

Camada de Aplicação

Introdução à camada de aplicação, aplicação Web e protocolo HTTP

Profa. Anelise Munaretto e Prof. Mauro Fonseca

DAINF - UTFPR

Material de Referência Bibliográfica

Kurose, J. F. & Ross, K. W. — *Redes de Computadores e a Internet: Uma Abordagem Top-Down*

https://gaia.cs.umass.edu/kurose_ross/

O que é a Camada de Aplicação: Fundamentos, Processos e Sockets

Quais Serviços o Transporte Oferece: TCP, UDP e TLS

Como a Web Funciona: HTTP, Mensagens e Conexões

Como Manter Estado e Acelerar: Cookies e Caching

Como a Web Evoluiu: HTTP/1.0 → HTTP/3

Síntese: Revisão e Ponte para a Aula 3

Visão Geral: Camada de Aplicação

Roteiro dos principais conceitos, protocolos e paradigmas da Camada de Aplicação:

Protocolos Fundamentais

- **Princípios de Aplicações:** Arquitetura Cliente-Servidor e requisitos de transporte.
- **Web & HTTP:** Métodos, cabeçalhos, persistência, cookies e cache.
- **Correio Eletrônico:** Envio via SMTP e acesso via IMAP/POP3.
- **DNS:** Infraestrutura hierárquica e resolução de nomes em endereços IP.

Paradigmas e Implementação

- **Redes P2P:** Escalabilidade descentralizada e compartilhamento distribuído (ex.: BitTorrent).
- **Streaming de Vídeo & CDNs:** DASH, controle de taxa adaptativa e caches de borda.
- **Programação com Soquetes:** Interface de rede (*Socket API*) usando TCP e UDP.

Objetivo da Camada

Permitir que processos residentes em sistemas finais distintos troquem mensagens estruturadas por meio da interface de transporte oferecida pelo sistema operacional.

O que é a Camada de Aplicação: Fundamentos, Processos e Sockets

Ecosistema de Aplicações de Rede

As aplicações de rede são a razão de existência da Internet. Elas impõem requisitos distintos de vazão, tolerância a perdas e atraso sobre a infraestrutura:

1. Dados e Web (Sem Perdas)

- **Web & Navegação:** HTTP/1.1, HTTP/2 e HTTP/3.
- **Mensageria & E-mail:** SMTP, IMAP e mensageiros instantâneos.
- **Acesso Remoto & Transferência:** SSH, SFTP e distribuição P2P (BitTorrent).
- **Motores de Busca:** Crawling em escala e indexação distribuída.

2. Tempo Real, Mídia e Nuvem

- **Streaming de Vídeo:** YouTube, Netflix (DASH, HLS sobre CDN).
- **Comunicação em Tempo Real:** VoIP, videoconferência (Zoom, Meet, WebRTC).
- **Jogos Multiusuário:** Interações com requisitos rígidos de latência.
- **Cloud Computing:** Microsserviços e APIs (AWS, Azure, GCP).

Implicação para a Camada de Transporte

A heterogeneidade dessas aplicações exige a escolha criteriosa do protocolo de transporte (TCP para integridade estrita vs. UDP ou QUIC para menor atraso).

Desenvolvimento de Software de Rede: O Princípio Fim-a-Fim

O Paradigma da Aplicação de Rede

Programas de usuário são desenvolvidos e executados **exclusivamente nos sistemas finais** (*hosts/end systems*), comunicando-se pela infraestrutura de rede por meio da interface do sistema operacional.



1. Núcleo Simples (Dumb Core)

Roteadores e comutadores não processam camadas de aplicação; executam apenas encaminhamento rápido de pacotes nas camadas inferiores (L1–L3).

2. Inovação Rápida na Borda

A inteligência está restrita aos sistemas finais. Novos serviços (ex.: HTTP, BitTorrent, WebRTC) são criados sem requerer qualquer alteração no hardware do núcleo.

Arquiteturas de Aplicação: Paradigmas Principais

A organização estrutural dos sistemas finais define como as mensagens são trocadas, a escalabilidade e o modelo de governança da aplicação:

1. Arquitetura Cliente-Servidor

Servidor (Always-on):

- Endereço IP fixo e permanente.
- Hospedado em data centers / nuvem para alta disponibilidade.

Clientes:

- Comunicação passiva sob demanda; não dialogam entre si diretamente.
- Podem ter IPs dinâmicos e conexão intermitente.
- **Exemplos:** Web (HTTP), E-mail, Banco de Dados.

2. Arquitetura Peer-to-Peer (P2P)

Pares (Peers):

- Comunicação direta e simétrica entre sistemas finais arbitrários.
- Sem dependência de um servidor central contínuo.

Propriedades:

- **Auto-escalabilidade:** Cada novo par traz capacidade de serviço ao mesmo tempo que demanda.
- Alta complexidade de gestão e trânsito NAT.
- **Exemplos:** BitTorrent, Blockchain.

Arquitetura Cliente-Servidor: Estrutura e Papéis

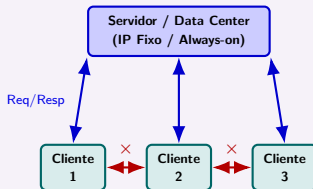
Servidor (Host Central)

- **Disponibilidade Contínua:** Equipamento sempre ativo (*always-on*).
- **Endereçamento:** IP estático/fixo e conhecido (ou associado a DNS).
- **Escalabilidade:** Hospedado em *data centers* ou nuvem para atender requisições em larga escala.

Clientes (Hosts Finais)

- **Iniciativa de Contato:** Solicitam serviços sob demanda.
- **Intermitência:** Podem desconectar e alternar IPs (endereço dinâmico).
- **Isolamento:** Não estabelecem comunicação direta entre si.

Topologia Estrela / Centralizada



Sem comunicação P2P direta

Ponto Único de Falha (SPOF)

A centralização simplifica a governança, mas exige balanceamento de carga para mitigar gargalos no servidor.

Arquitetura Peer-to-Peer (P2P): Descentralização

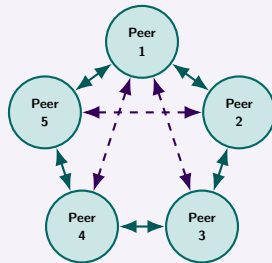
Propriedades do Modelo P2P

- **Descentralização Estrita:** Não exige servidores centrais dedicados ou permanentemente ativos.
- **Papéis Simétricos (Servents):** Cada par atua simultaneamente como cliente (consome) e servidor (fornece).
- **Auto-escalabilidade (Self-scalability):** A capacidade total de envio do sistema cresce proporcionalmente à entrada de novos pares.
- **Alta Volatilidade (Churn):** Nós entram e saem dinamicamente, alternando endereços IP e disponibilidade.

Aplicação Clássica: BitTorrent

Distribuição massiva de arquivos em blocos paralelos (ex.: ISOs de distribuições Linux, atualizações de jogos e patches).

Topologia em Malha (Mesh)



Desafio Técnico

Transposição de NAT e firewalls para conexões diretas de entrada.

Processos e Comunicação na Rede

Em redes, a comunicação não ocorre diretamente entre máquinas físicas, mas sim entre **processos** (instâncias de programas em execução nos sistemas finais):

Escopos de Comunicação

- **Comunicação Local (Mesmo Host):** Gerenciada pelo SO via *Inter-Process Communication (IPC)* (ex.: *pipes*, memória compartilhada, Unix sockets).
- **Comunicação em Rede (Hosts Distintos):** Realizada exclusivamente pela troca de **mensagens** estruturadas sobre a pilha de protocolos.

Papéis na Sessão de Diálogo

- **Processo Cliente:** O processo que toma a iniciativa de **abrir a conexão** e enviar a requisição.
- **Processo Servidor:** O processo passivo que aguarda ser contactado para fornecer a resposta/serviço.

Nota Arquitetural: Papéis Dinâmicos em Redes P2P

Os papéis de cliente e servidor são atribuídos por **sessão de comunicação individual**, e não pela máquina. Em aplicações P2P (como o BitTorrent), o mesmo processo atua simultaneamente como **cliente** (ao baixar um bloco de um par) e como **servidor** (ao enviar outro bloco para um par diferente).

Endereçamento de Processos na Rede

Para que uma mensagem chegue corretamente a uma aplicação específica, a rede precisa de uma chave de endereçamento hierárquica em dois níveis:

1. Endereço IP (Nível Host)

- **Papel:** Identifica a interface de rede do dispositivo (*host*) no planeta.
- **Camada:** Opera na **Camada de Rede** (L3).
- **Formato:** 32 bits (IPv4) ou 128 bits (IPv6).
- *Apenas o IP não basta: uma mesma máquina roda múltiplos serviços simultaneamente.*

2. Número de Porta (Nível Processo)

- **Papel:** Identifica o processo/soquete específico dentro daquele host.
- **Camada:** Opera na **Camada de Transporte** (L4).
- **Formato:** Inteiro de 16 bits (0 a 65.535).
- **Padronização:** *Well-known ports* (0–1023) reservadas pela IANA para serviços padrão.

Identificador Completo do Processo: <Endereço IP : Número de Porta>

HTTP (Web): Porta 80
SMTP (E-mail): Porta 25

HTTPS (Web Segura): Porta 443
IMAP (E-mail): Porta 143

DNS (Nomes): Porta 53
FTP (Controle): Porta 21

SSH (Login Remoto): Porta 22
DHCP (Configuração): Porta 67/68

Soquetes: A Interface de Programação de Rede

O Soquete como Interface (API)

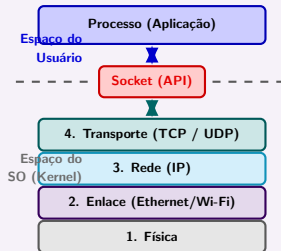
Ponto de contato entre o processo de aplicação (espaço do usuário) e a pilha de protocolos de rede (espaço do núcleo do SO):

- **Analogia da Porta:** O processo joga mensagens através do soquete e confia no SO/infraestrutura para a entrega remota.
- **Identificação:** Vinculado a uma porta local e ao IP da máquina.

Divisão de Responsabilidades

- **Desenvolvedor:** Controle total da lógica de aplicação, formato da mensagem e escolha do protocolo (TCP ou UDP).
- **Sistema Operacional:** Controle do transporte, temporizadores, janelas de fluxo/congestionamento e envio físico.

Arquitetura do Soquete



Definição de um Protocolo de Aplicação

O que um Protocolo de Aplicação Define?

Regula formalmente como processos trocam mensagens pela rede:

- **Tipos de Mensagens:** Requisições (*requests*) vs. Respostas (*responses*).
- **Sintaxe & Semântica:** Estrutura dos campos, delimitadores e o significado de cada dado.
- **Regras de Diálogo:** Quando e como um processo envia ou responde a mensagens.

1. Protocolos Abertos (RFCs)

- Padronizados pela **IETF** via **RFCs**.
- Permitem total interoperabilidade entre diferentes fabricantes e clientes.
- **Exemplos:** HTTP, DNS, SMTP.

2. Protocolos Proprietários

- Especificação fechada e controlada por uma empresa específica.
- Sem garantia de interoperabilidade pública.
- **Exemplos:** Skype original, WhatsApp.

Aplicação vs. Protocolo de Aplicação

A **Web** é a aplicação distribuída; o **HTTP** é o protocolo de comunicação aberto que a viabiliza.

Requisitos das Aplicações para o Transporte

Ao seleccionar ou projetar o protocolo de transporte, a aplicação impõe restrições em quatro dimensões principais:

1. Integridade de Dados (Perdas)

- **Transferência Confiável:** Exigem 100% de integridade; perda zero aceita (ex.: Web, e-mail, arquivos).
- **Tolerantes a Perdas:** Pequenas perdas são imperceptíveis ou aceitáveis (ex.: áudio/vídeo, VoIP).

2. Temporização (Latência / Jitter)

- **Sensíveis a Atraso:** Exigem atraso fim-a-fim estritamente baixo (ex.: jogos online, teleconferência).
- **Insensíveis a Atraso:** Operam sem prejuízo sob latências variáveis (ex.: e-mail, download HTTP).

3. Vazão (Throughput)

- **Sensíveis à Vazão:** Exigem taxa mínima de transmissão para funcionar (ex.: streaming HD/4K).
- **Aplicações Elásticas:** Adaptam-se a qualquer banda disponível (ex.: transferências SCP/HTTP).

4. Segurança

- **Confidencialidade:** Criptografia do canal.
- **Autenticação e Integridade:** Prevenção contra adulteração em trânsito (ex.: TLS/HTTPS).

A Internet Oferece Garantias de Tempo e Vazão?

A arquitetura IP/TCP padrão é de **melhor esforço (best-effort)**: não fornece garantias rígidas de atraso mínimo nem de vazão reservada.

Requisitos de Aplicações de Rede Comuns

Matriz de Requisitos de Transporte das Aplicações

Aplicação	Tolerância a Perdas	Demanda de Vazão	Sensível a Atraso?
Transferência de Arquivos	Nula (<i>Loss-intolerant</i>)	Elástica	Não
Correio Eletrônico (E-mail)	Nula (<i>Loss-intolerant</i>)	Elástica	Não
Documentos Web (HTTP)	Nula (<i>Loss-intolerant</i>)	Elástica	Não (poucos s)
Telefonia IP / Vídeo em Tempo Real	Sim (<i>Loss-tolerant</i>)	Fixa (5 kbps–5 Mbps)	Sim (≤ 150 ms)
Streaming de Vídeo Armazenado	Sim (<i>Loss-tolerant</i>)	Variável (10 kbps–25 Mbps)	Parcial (alguns s)
Jogos Interativos em Rede	Sim (<i>Loss-tolerant</i>)	Baixa (dezenas de kbps)	Sim (≤ 100 ms)
Mensageria Instantânea	Nula (<i>Loss-intolerant</i>)	Elástica	Parcial (poucos s)

Aplicações Elásticas

Consomem tanta banda quanto o enlace disponibilizar; adaptam sua taxa dinamicamente via controle de congestionamento (ex.: TCP).

Aplicações de Tempo Real

Priorizam baixa latência e baixo *jitter* sobre confiabilidade estrita; frequentemente usam *buffers* de reprodução para absorver variações de atraso.

**Quais Serviços o Transporte
Oferece: TCP, UDP e TLS**

Serviços de Transporte da Internet: TCP vs. UDP

1. TCP (Transmission Control Protocol)

- **Orientado à Conexão:** Exige estabelecimento prévio de sessão (*3-way handshake*).
- **Transporte Confiável:** Garante entrega ordenada e sem perdas (ACKs, retransmissões).
- **Controle de Fluxo:** Ajusta a taxa para não afogar o buffer do receptor.
- **Controle de Congestionamento:** Regula o envio em função da saturação do núcleo da rede.

2. UDP (User Datagram Protocol)

- **Sem Conexão (Connectionless):** Envio direto sem fase inicial de sincronização.
- **Melhor Esforço (Unreliable):** Não assegura entrega, integridade estrita ou ordenação.
- **Sem Controles de Fluxo/Rede:** Emite à taxa que a aplicação determinar.
- **Baixa Sobrecarga (Overhead):** Cabeçalho mínimo (8 bytes) e atraso de início nulo.

O que a Internet Não Fornece Nativamente (Nem TCP, Nem UDP)

Sem Garantia de Atraso Máximo: Não assegura limites estritos de temporização ou *jitter*. | **Sem Garantia de Vazão Mínima:** Não reserva taxa de transmissão dedicada ao longo da rota.

Mapeamento: Protocolos de Aplicação vs. Transporte

Matriz de Serviços de Aplicação e Escolha de Transporte

Aplicação	Protocolo da Camada de Aplicação	Transporte Subjacente
Correio Eletrônico	SMTP [RFC 5321]	TCP
Acesso Remoto Seguro	SSH [RFC 4251] / Telnet	TCP
Navegação Web	HTTP/1.1, HTTP/2 e HTTP/3	TCP / QUIC (UDP)
Transferência de Arquivos	FTP [RFC 959] / SFTP	TCP
Resolução de Nomes	DNS [RFC 1035]	UDP ou TCP
Streaming Multimídia	DASH, HLS ou RTP [RFC 3550]	TCP ou UDP
Telefonia IP / VoIP	SIP [RFC 3261], RTP ou WebRTC	UDP tipicamente (ou TCP)

Por que escolher TCP?

- **Integridade Estrita:** Dados de texto, páginas e binários não toleram perda de bytes.
- **Travessia de Firewalls:** Portas como 80 (HTTP) e 443 (HTTPS) são amplamente liberadas.

Por que escolher UDP / QUIC?

- **Sem Atraso por Retransmissão:** Em voz/vídeo ao vivo, pacotes atrasados perdem utilidade.
- **Conexão Rápida:** DNS e QUIC evitam o custo de múltiplos RTTs de sincronização inicial.

Segurança na Camada de Transporte: TLS/SSL

Limitações Nativas do TCP

O TCP clássico transmite dados em texto claro (*plaintext*), expondo a comunicação a interceptação (*sniffing*) e adulteração (*tampering*).

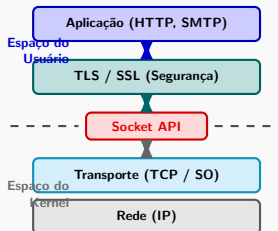
Serviços Oferecidos pelo TLS

- **Confidencialidade:** Criptografia simétrica dos dados em trânsito.
- **Integridade:** Códigos de autenticação de mensagem (MAC/HMAC).
- **Autenticação:** Certificados digitais X.509 (chave pública/servidor).

Localização na Pilha

O TLS opera no **espaço do usuário** (camada de aplicação), provendo uma API de soquete seguro sobre o TCP clássico (ex.: HTTPS na porta 443).

Posicionamento do TLS



Como a Web Funciona: HTTP, Mensagens e Conexões

Web e HTTP: Estrutura de Objetos e URLs

O Modelo de Página Web

Uma página Web não é um arquivo único, mas um conjunto estruturado de **objetos** digitais:

- **Arquivo Base (HTML Base):** Contém a estrutura da página e referencia objetos externos através de *tags* (ex.: ``, `<link>`, `<script>`).
- **Objetos Referenciados:** Imagens (JPEG, PNG), folhas de estilo (CSS), scripts (JavaScript), fontes e vídeos.
- **Endereçamento Único:** Cada objeto é acessível através de uma **URL** (*Uniform Resource Locator*).

Anatomia Estrutural de uma URL

`http://www.universidade.br/departamento/foto.jpg`

↑
Protocolo
(Scheme)

↑
Nome do Host
(Hostname / DNS)

↑
Caminho do Recurso
(Path name)

Resolução de Recursos

O cliente HTTP usa o **DNS** para mapear o *hostname* em IP e solicita o *path* via requisição GET.

Múltiplas Conexões

Para exibir uma página complexa, o navegador executa requisições separadas para **cada objeto** referenciado no HTML.

HTTP: Protocolo de Transferência de Hipertexto

Fundamentos do Protocolo

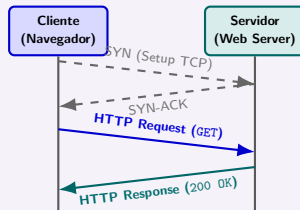
Protocolo da camada de aplicação da Web (padronizado pela IETF via RFCs):

- **Modelo Cliente-Servidor:** Navegador (*cliente*) requisita objetos; servidor Web (*Apache, Nginx*) envia as respostas.
- **Transporte Subjacente:** Utiliza **TCP** (porta padrão 80 para HTTP e 443 para HTTPS).

Protocolo Sem Estado (Stateless)

- O servidor não mantém informações ou histórico sobre requisições passadas do cliente.
- **Vantagem:** Projeto de servidor simples e altamente escalável.
- **Estado na Web:** Gerenciado externamente via **Cookies** e **Tokens** de sessão.

Diálogo Cliente-Servidor



Evolução do Transporte

HTTP/1.1 e HTTP/2 rodam sobre TCP; HTTP/3 utiliza **QUIC** sobre UDP para eliminar atrasos de HOL (*Head-of-Line*).

HTTP: Transporte Subjacente e Natureza Stateless

1. Uso da Camada de Transporte (TCP)

- **Conexão Confiável:** O HTTP delega a integridade dos dados ao TCP (porta padrão 80 ou 443 no HTTPS).
- **Ciclo de Vida:** O cliente abre um soquete TCP → estabelece o *handshake* → troca mensagens HTTP (*req/resp*).
- **Transparência:** O HTTP não precisa implementar confirmações (ACKs) ou retransmissões internas.

2. Natureza Sem Estado (Stateless)

- **Definição:** O servidor processa cada requisição de forma isolada, sem registrar histórico de interações anteriores do cliente.
- **Independência:** Duas requisições consecutivas do mesmo cliente são tratadas como eventos totalmente não correlacionados.

Por que um Projeto Stateless?

- **Alta Escalabilidade:** Servidores não alocam memória para manter contexto de milhões de clientes simultâneos.
- **Tolerância a Falhas:** Se um servidor cai e reinicia, não há estado local a ser reconstruído ou sincronizado.

Como a Web Mantém Estado?

Para autenticação e carrinhos de compras, o estado é mantido externamente por meio de **Cookies** e **Tokens** tráfegados nos cabeçalhos HTTP.

Conexões HTTP: Evolução e Persistência

Evolução dos Modos de Conexão na Web

Versão	Conexão	Padrão	Comportamento / Impacto
HTTP/1.0	Não Persistente	Sim	1 objeto por conexão TCP (2 RTTs + <i>slow start</i> por recurso).
HTTP/1.1	Persistente	Sim	Reutiliza o socket TCP (<i>Keep-Alive</i>); suporta <i>pipelining</i> .
HTTP/2	Persistente	Sim	Multiplexação de fluxos binários em uma única conexão TCP.
HTTP/3	Persistente	Sim	Opera sobre QUIC (UDP) , eliminando o bloqueio de cabeça de fila (<i>HOL</i>).

1. Não Persistente (HTTP/1.0)

- Cada objeto web exige um novo *3-way handshake* TCP.
- Alto consumo de buffers no SO e latência cumulativa severa em páginas ricas em objetos.

2. Persistente (HTTP/1.1+)

- O socket permanece aberto após a resposta; múltiplos objetos trafegam na mesma conexão.
- **Ganho:** Elimina RTTs adicionais de setup e aproveita a janela TCP já expandida.

Vantagem Fundamental da Persistência

A persistência evita a penalidade do **TCP Slow Start** a cada objeto referenciado, reduzindo significativamente o tempo total de carregamento da página (*Page Load Time*).

Tempo de Resposta: Análise do RTT

Definição de RTT (Round-Trip Time)

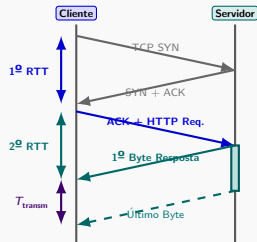
Tempo necessário para um pacote pequeno viajar do cliente ao servidor e retornar (inclui propagação, filas e processamento).

Modelo Analítico de Atraso (1 Objeto)

Tempo Total = $2 \times \text{RTT} + T_{\text{transmissão}}$

- **1º RTT**: Estabelece conexão (*TCP Handshake*).
- **2º RTT**: Requisição HTTP (GET) + chegada do 1º byte da resposta.
- T_{transm} : Injeção de bits no enlace (L/R).

Diagrama de Atraso por Objeto



HTTP Não Persistente: Sobrecarga em Múltiplos Objetos

Cenário: HTML Base + 10 Imagens Referenciadas

Como o socket TCP fecha a cada resposta, o custo de $2 \times \text{RTT}$ é pago para cada um dos 11 objetos:

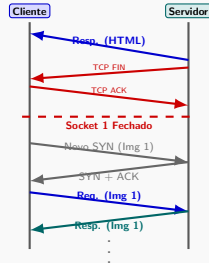
Gargalos do Modelo

- **Latência Cumulativa:** $22 \times \text{RTT} + 11 \times T_{\text{transm}}$ no modo sequencial puro.
- **Sobrecarga de SO:** 11 conexões criadas e destruídas (FIN/ACK), consumindo tabelas de estado.
- **Janela Congestionada:** Cada conexão zera o *Slow Start*, impedindo o link de atingir a capacidade máxima.

Tentativas de Mitigação no HTTP/1.0

Navegadores abriam conexões TCP em paralelo (5 a 8 simultâneas), reduzindo o tempo percebido à custa de saturar a rede e o servidor.

Diagrama de Teardown e Reabertura



HTTP Persistente com Pipelining (HTTP/1.1)

Cenário: HTML Base + 10 Imagens Referenciadas (Comparação Direta)

Enquanto o HTTP/1.0 exigia 22 RTTs (11 conexões TCP serials), o HTTP/1.1 reutiliza o mesmo socket e envia requisições em lote:

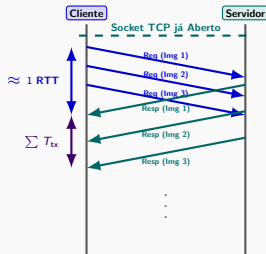
Ganhos de Desempenho

- **Setup Nulo (Ganha 1 RTT por objeto):** O socket já está aberto; elimina 10 handshakes TCP adicionais.
- **Requisições em Lote:** O cliente despacha os 10 GETs assim que analisa o HTML, sem esperar as respostas.
- **Custo Total:** Apenas ≈ 1 RTT para todas as imagens
+ $\sum \tau_{\text{transmissão}}$.

Limitação: Bloqueio HOL (Head-of-Line)

As respostas devem retornar na ordem exata das requisições (FIFO). Se a Img 1 atrasar, trava a fila inteira no TCP (resolvido no HTTP/2 com multiplexação).

Diagrama Espaço-Tempo (Pipelining)



Mensagem de Requisição HTTP: Estrutura

Texto Plano (ASCII): As mensagens HTTP são projetadas para serem lidas por humanos, o que facilita a análise e depuração em redes

Exemplo de Requisição (Método GET)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Connection: keep-alive\r\n
\r\n
```

1. Linha de Requisição (1ª linha):

- **Método:** GET, POST, HEAD, PUT, etc
- **URL:** O caminho do objeto solicitado
- **Versão:** A versão do protocolo em uso

2. Linhas de Cabeçalho (Headers):

- **Pares Nome:** Valor com informações do cliente
- **\r\n:** *Carriage Return + Line Feed* (Sinaliza o fim de cada linha)
- **Linha Extra (\r\n):** Uma linha totalmente em branco indica ao servidor que os cabeçalhos acabaram

Estrutura Formal da Mensagem de Requisição

O formato geral de uma requisição HTTP é rigorosamente definido. Cada espaço e quebra de linha tem uma função vital para o *parser* do servidor

método	sp	URL	sp	versão	cr	lf	Linha de Requisição
nome do campo:		sp	valor		cr	lf	
nome do campo:		sp	valor		cr	lf	Linhas de Cabeçalho
					cr	lf	
					cr	lf	Linha em Branco (Fim do Cabeçalho)
Corpo da Entidade (<i>Entity Body</i>)							Dados Úteis (Ex: Formulários no método POST)

- **sp (space):** Espaço em branco obrigatório para separar os comandos
- **cr lf (carriage return + line feed):** Sequência de controle que define o fim exato de uma linha no fluxo de bytes do TCP
- **Corpo da Entidade:** Fica vazio nas requisições GET, mas é utilizado no POST para enviar dados (como senhas e formulários) ao servidor

Enviando Dados ao Servidor: POST vs GET

Entrada do Usuário: Como o navegador envia os dados preenchidos em formulários ou caixas de busca para o servidor web?

Método POST (Corpo da Mensagem)

- Os dados viajam encapsulados no **Corpo da Entidade** (*Entity Body*)
- A URL solicitada permanece limpa
- **Ideal para:** Upload de arquivos, envio de formulários extensos e dados sensíveis

Visão no Socket:

```
POST /login.php HTTP/1.1
Host: www.site.com
[... cabeçalhos ...]
\r\n
user=aluno&senha=secreta
```

Método GET (Anexado na URL)

- O Corpo da Entidade fica **vazio**
- Os dados são injetados diretamente na URL (após o caractere ?)
- **Ideal para:** Buscas simples e filtros, onde o estado pode ser salvo nos favoritos

Visão no Socket:

```
GET /busca?termo=redes&aula=2 HTTP/1.1
Host: www.site.com
[... cabeçalhos ...]
\r\n
(Corpo Vazio)
```

- **Aviso de Privacidade:** O método GET expõe os parâmetros na barra de endereços do navegador, no histórico e nos logs de roteadores. Nunca deve ser usado para senhas

Evolução dos Métodos HTTP

Além da navegação básica, a evolução do protocolo introduziu métodos fundamentais para tornar a web interativa e programável

Base Tradicional (HTTP/1.0)

- GET: Solicita a transferência de um documento (HTML, imagens, etc.)
- POST: Envia dados para processamento no servidor (no Corpo da Entidade)
- HEAD: Solicita **apenas** as linhas de cabeçalho da resposta
 - *Uso Prático:* Ferramentas de busca e depuração usam para verificar se um link está quebrado (ex: erro 404) ou se o arquivo foi modificado, **sem gastar banda** baixando o objeto inteiro

Adições Avançadas (HTTP/1.1)

- PUT: Faz o *upload* de um objeto para o diretório e arquivo especificados na URL
- DELETE: Solicita ao servidor a exclusão do arquivo especificado na URL

Contexto Moderno:

Embora presentes desde o HTTP/1.1, os métodos PUT e DELETE ganharam adoção massiva com a arquitetura de **APIs REST**, onde a URL representa um recurso (ex: `/alunos/10`) e o método dita a ação (Atualizar/Apagar)

Mensagem de Resposta HTTP: Estrutura

A Resposta do Servidor: Assim como a requisição, a resposta é formatada em texto plano, dividida entre metadados e o conteúdo real

Exemplo de Resposta

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2021 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2020 17:00:00\r\n
Content-Length: 2652\r\n
Content-Type: text/html\r\n
Connection: Keep-Alive\r\n
\r\n
[... dados do objeto requisitado ...]
[... código HTML, imagem JPEG, etc ...]
```

1. Linha de Status (1ª linha):

- **Versão:** Protocolo utilizado
- **Código:** Status numérico (ex: 200, 404)
- **Frase:** Explicação legível do código

2. Cabeçalhos (Headers):

- **Server:** O software web usado pelo host (útil para Fingerprinting)
- **Content-Length:** Tamanho exato do arquivo em bytes
- **Content-Type:** O tipo MIME do arquivo. Diz ao navegador se ele deve renderizar um HTML, tocar um MP3 ou baixar um ZIP

Códigos de Status HTTP: Classes e Exemplos

O código de status (3 dígitos) aparece na primeira linha da resposta. O **primeiro dígito** define a classe (categoria) do resultado

2xx: Sucesso (Ação concluída)

- **200 OK:** Requisição processada com sucesso. O objeto solicitado segue no corpo da mensagem

3xx: Redirecionamento

- **301 Moved Permanently:** O objeto mudou de endereço. A nova URL vai no cabeçalho Location
- **304 Not Modified:** O cache local do cliente ainda é válido. O servidor não envia o objeto novamente (economiza banda)

4xx: Erro do Cliente (Sintaxe/URL)

- **400 Bad Request:** O servidor não compreendeu a requisição (ex: erro de formatação)
- **404 Not Found:** O documento solicitado não existe neste servidor

5xx: Erro do Servidor

- **500 Internal Server Error:** Falha na aplicação do servidor (ex: erro em um script PHP ou banco de dados)
- **505 HTTP Version Not Supported:** O servidor não suporta a versão exigida pelo cliente

Experimentando o HTTP na Prática (Via Terminal)

Como o HTTP original é baseado em texto plano (ASCII), podemos assumir o papel do navegador e “conversar” diretamente com o servidor usando o **Telnet**

Passo 1: Estabelecer Conexão TCP

- Abra o terminal e conecte-se à porta 80

Comando (Linux/Windows)

```
$ telnet gaia.cs.umass.edu 80
```

Passo 2: Enviar a Requisição HTTP

- Digite o método, a URL e os cabeçalhos
- Pressione **Enter DUAS** vezes para criar a linha em branco (fim da requisição)

Digite exatamente:

```
GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu
<Enter>
<Enter>
```

Passo 3: Analisar o Retorno

- O servidor responderá exibindo a linha de status (200 OK), os cabeçalhos e o código HTML direto na tela do seu terminal

Nota sobre a Web Moderna (HTTPS)

Este experimento funciona perfeitamente neste servidor acadêmico (porta 80). Porém, em sites modernos (Google, bancos, etc.), a porta padrão é a **443 (HTTPS)**. O Telnet puro falharia nesses casos, pois não consegue realizar o *handshake* de criptografia TLS exigido antes do envio do texto

Como Manter Estado e Acelerar: Cookies e Caching

Cookies: Gerenciando o Estado (O “Crachá” da Web)

O Problema da Amnésia: O protocolo HTTP é *stateless* (sem estado). O servidor não lembra de requisições passadas. Para ele, cada clique vem de um “desconhecido”

A Analogia do Crachá

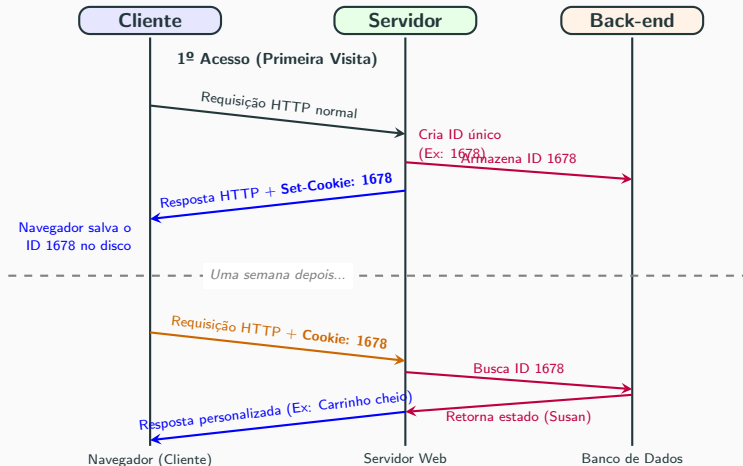
- **1ª Visita:** Você chega num prédio (site). A portaria te cadastra e te entrega um **crachá com um número** (ID único)
- **Próximas Portas:** Para não ter que se apresentar a cada sala que entra, você apenas **mostra o crachá**
- **Resultado:** O sistema olha o número, consulta os registros e diz: *“Ah, é a Susan, e ela tem 3 itens no carrinho de compras”*

Os 4 Componentes Técnicos:

1. **Resposta HTTP (Emitindo o crachá):** O servidor envia o ID para o cliente usando a linha de cabeçalho **Set-Cookie: 1678**
2. **Arquivo Local (O porta-crachá):** O navegador (Chrome/Firefox) armazena esse ID silenciosamente no disco do usuário
3. **Requisição HTTP (Mostrando o crachá):** Em acessos futuros, o navegador anexa a linha **Cookie: 1678** na mensagem
4. **Banco de Dados (O sistema de controle):** O *back-end* lê o ID recebido e restaura o estado da sessão

Funcionamento dos Cookies na Prática

O protocolo HTTP continua sendo *stateless*. O que mantém o estado (a sessão) é a aplicação, usando o cabeçalho HTTP como mensageiro do ID



Cookies: Aplicações Práticas e Privacidade

Os cookies resolveram a “amnésia” do HTTP, permitindo a web interativa, mas criaram o maior debate de privacidade da história da Internet

Aplicações (O Lado Útil):

- **Autorização:** Mantém o usuário *logado* no webmail ou no portal acadêmico sem pedir senha a cada clique
- **Carrinhos de Compras:** O servidor lembra os itens escolhidos enquanto o cliente navega por outros departamentos
- **Recomendações:** Personalização da vitrine com base nos cliques anteriores daquela sessão
- **Estado da Sessão:** Gerencia formulários complexos de múltiplas etapas (ex: declaração de imposto)

O Custo: Rastreamento

Ao cruzar o ID do cookie com um cadastro (nome/e-mail), o servidor deixa de ver um “anônimo” e mapeia a identidade real a todo o comportamento passado

O Grande Vilão: Cookies de Terceiros

- Redes de publicidade inserem elementos invisíveis (scripts/anúncios) em milhares de sites independentes
- Elas leem o seu “crachá” através das fronteiras dos sites, construindo um perfil de consumo global
- *Contexto:* É exatamente para frear isso que legislações como a **LGPD** (no Brasil) foram criadas

Caches Web (Servidor Proxy)

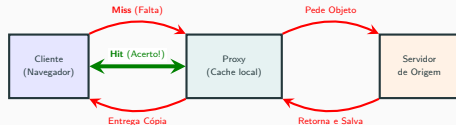
Objetivo: Satisfazer a requisição do cliente sem envolver o servidor de origem, reduzindo o tempo de resposta e poupando a banda do link externo

A Dupla Personalidade:

- O Proxy atua simultaneamente como **Servidor** (para o cliente local) e como **Cliente** (para o servidor de origem)

O Fluxo de Requisição:

1. O navegador aponta para o Proxy e envia todas as requisições HTTP a ele
2. **Cache Hit:** Objeto armazenado localmente
⇒ o Proxy retorna de imediato
3. **Cache Miss:** O Proxy requisita à origem, **salva uma cópia** no disco e entrega ao cliente



Benefícios do Cache

- Reduz o atraso para o usuário final
- Reduz o tráfego nos links de acesso (ex: UTFPR)

Por que utilizar Caching na Web?

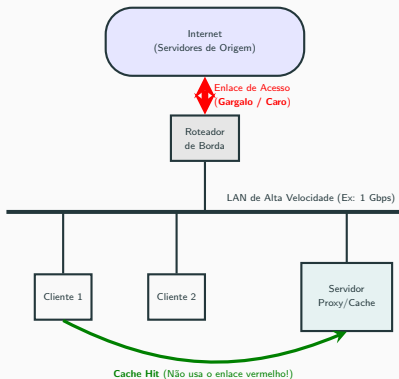
A instalação de um Servidor Proxy local é uma das soluções de engenharia mais elegantes para resolver problemas de latência e custo financeiro em redes

Instalação e Posicionamento:

- Mantido pelo ISP local (redes corporativas, universidades ou provedores residenciais)
- Fica “próximo” aos clientes, dentro da rede de alta velocidade (LAN)

Os 3 Grandes Benefícios:

- **Latência (Tempo de Resposta):** Reduz o tempo de espera do usuário quase a zero em caso de acerto (*Hit*), pois a requisição não viaja pela Internet
- **Redução de Tráfego:** Alivia drasticamente a carga no enlace de acesso (que é o link mais lento e mais caro de manter)
- **Desempenho Global:** Evita que os servidores de origem colapsem, democratizando a entrega de conteúdo pesado para as massas



Exemplo Prático: A Matemática da Economia

O Cenário: Uma instituição possui uma LAN de **10 Gbps** e um **Enlace de Acesso de 100 Mbps** para a Internet

- Taxa de requisições: 100 objetos/segundo
- Tamanho médio do objeto: 1 Megabit (Mb)
- Atraso médio da Internet (ida e volta): 2 segundos

Cenário A: Sem Cache

1. Uso da LAN: 100 Mbps (1% de 10 Gbps → Atraso local nulo)

2. Tráfego no Enlace (Gargalo):

$$\frac{100 \text{ req/s} \cdot 1 \text{ Mb}}{100 \text{ Mbps}} = 100\% \text{ de uso}$$

Resultado (Colapso): Com a utilização do enlace no limite (100%), a fila no roteador cresce rapidamente. O atraso total passa de **minutos**, travando a navegação

A Grande Lição: O proxy evitou a necessidade de comprar um link mais rápido, resolvendo o gargalo de tráfego via *software*

Cenário B: Com Cache (Acerto de 40%)

1. Tráfego Direcionado: 40% dos objetos são entregues localmente. Apenas **60%** vão para a Internet (60 req/s)

2. Nova Utilização do Enlace:

$$\frac{60 \text{ req/s} \cdot 1 \text{ Mb}}{100 \text{ Mbps}} = 60\% \text{ de uso}$$

Resultado (Sucesso): A fila do roteador esvazia.

- Atraso = $0.4 \cdot (\text{Instantâneo}) + 0.6 \cdot (2s)$
- Atraso total cai para $\approx 1,2$ segundos!

Caches Web: O Mecanismo do GET Condicional

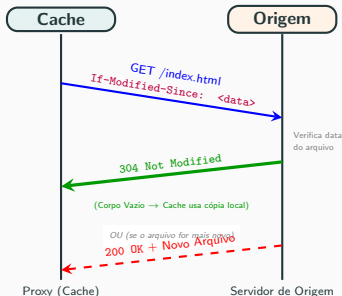
O Problema da Atualização: Como o cache garante que não está entregando uma versão velha (obsoleta) do site ao usuário?

A Lógica da Validação:

- **A Pergunta:** O cache envia uma requisição HTTP normal, mas adiciona o cabeçalho **If-Modified-Since:**
<data>
- **O Significado:** “Servidor, só me envie o arquivo se ele foi alterado **DEPOIS** desta data”

As Duas Respostas Possíveis

- **Não Mudou:** O servidor responde 304 Not Modified. O corpo da resposta vem **vazio** (economizando totalmente a banda do link)
- **Mudou:** O servidor responde 200 OK e anexa o novo arquivo no corpo da mensagem



Como a Web Evoluiu: **HTTP/1.0 → HTTP/3**

A Motivação para a Evolução do HTTP

A Web deixou de ser uma coleção de textos simples para se tornar uma plataforma de aplicações complexas, exigindo um protocolo cada vez mais rápido

O Limite Físico (Latência)

A velocidade da luz é um limite intransponível. Como não podemos fazer os dados viajarem mais rápido do que isso, a evolução foca em **reduzir o número de idas e voltas (RTT)** na rede

O Gargalo: HoL Blocking

Head-of-Line Blocking (Bloqueio de Cabeça de Fila): Ocorre quando um único pacote perdido ou atrasado na rede paralisa toda a fila de objetos que vinham logo atrás dele

A Linha do Tempo da Solução:



A Era do Texto (ASCII): HTTP/1.0 e HTTP/1.1

As primeiras gerações do protocolo focaram em otimizar o uso da conexão TCP subjacente para lidar com páginas web cada vez mais ricas

HTTP/1.0 (1996): O Gargalo do TCP

- **Conexões Efêmeras:** Abre e fecha um novo socket TCP para **cada** recurso solicitado (HTML, imagens, scripts)
- **O Problema:** O custo do *Handshake* triplo (SYN, SYN-ACK, ACK) repete-se indefinidamente
- **Resultado:** Latência extrema. Em uma página com 10 imagens, paga-se o “pedágio” do TCP 10 vezes

HTTP/1.1 (1997): Persistência

- **Keep-Alive:** A conexão TCP permanece aberta (*persistente*) e é reutilizada para múltiplos recursos
- **Pipelining:** Permite enviar vários pedidos em lote, sem esperar pela resposta do anterior

A Falha: HoL Blocking na Aplicação

Mesmo com *pipelining*, as respostas no HTTP/1.1 **devem voltar na mesma ordem** dos pedidos. Se o Recurso 1 (uma consulta pesada a um banco de dados) demorar, o Recurso 2 (uma imagem leve) fica bloqueado na fila do servidor esperando a sua vez

A Revolução Binária: Pilares do HTTP/2

O HTTP/2 manteve a semântica (GET, POST), mas reconstruiu completamente o formato de transporte para extrair o máximo do TCP

1. Framing Binário

As mensagens são divididas em *frames* binários (ex: *Frame* de Dados, *Frame* de Cabeçalho). Isso facilita o *parsing* para a máquina

2. Multiplexação Total

O fim do *HoL Blocking* na aplicação: pedidos/respostas simultâneos numa **única conexão TCP**, intercalando *frames* sem bloquear a fila

3. Compressão de Cabeçalhos (HPACK)

No HTTP/1.1, reenviar *Cookie/User-Agent* desperdiçava banda. O HPACK usa dicionários dinâmicos e envia apenas o que mudou

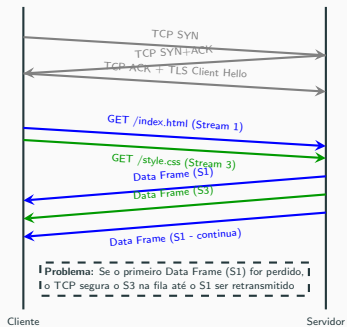
4. Server Push (deprecated no HTTP/2 moderno)

O servidor envia recursos proativamente. Se o cliente pede o `index.html`, o servidor já “empurra” o `style.css` junto, sabendo que ele será necessário, poupando 1 RTT. (Removido do padrão no RFC 9113 e desativado nos navegadores)



Única Conexão TCP (Fluxo Intercalado)

Fluxo de Operação: HTTP/2



Anatomia da Mensagem: Frames e Streams

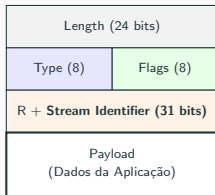
No HTTP/2, a unidade básica de comunicação é o **Frame**. Uma mensagem lógica (ex: um GET ou uma Resposta) é decomposta em um ou mais desses quadros

O que é um Stream?

É um fluxo bidirecional de *frames* dentro de uma conexão TCP

- Cada stream possui um **ID único**
- Permite que o cliente e servidor intercalem quadros de diferentes arquivos

Estrutura de um Frame (9 bytes de Header)



← O "R" do fluxo

Tipos Principais de Frames

- **HEADERS:** Contém os cabeçalhos HTTP (comprimidos com HPACK)
- **DATA:** Contém o corpo da mensagem (o arquivo em si)
- **SETTINGS/PRIORITY:** Controle da conexão

Por que Binário?

Diferente do HTTP/1.1 (texto), o receptor sabe exatamente quantos bytes ler para cada campo. Isso reduz drasticamente o uso de CPU e memória para processar a requisição. O "R" significa Reservado para uso futuro (1 bit)

A Mudança de Paradigma: HTTP/3 e QUIC

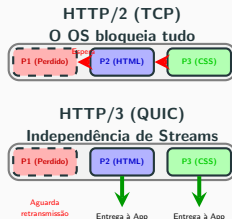
O HTTP/2 resolveu o bloqueio na Aplicação, mas o problema desceu para a **Camada de Transporte**. Se o TCP perde um pacote na rede, o Sistema Operacional bloqueia **todos** os *streams* do HTTP/2 até a retransmissão chegar

O Abandono do TCP

O HTTP/3 abandona o TCP em favor do protocolo **QUIC**, que roda sobre o **UDP**. O controle de congestionamento e a confiabilidade passam a ser implementados no espaço do usuário (*userspace*), fora do *kernel*

Segurança e Velocidade Nativa

O **TLS 1.3** está embutido no protocolo. O estabelecimento da conexão e da criptografia ocorre simultaneamente em apenas **1-RTT** (ou **0-RTT** para reconexões)



- **Resumo:** A perda de um pacote UDP afeta **apenas** o arquivo correspondente. Os outros *streams* continuam fluindo sem interrupções (*Zero HoL Blocking*)

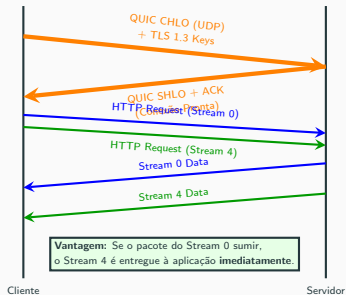
Segurança: O Salto do HTTP/3

- **HTTP/1.1 e 2.0:** A segurança (TLS) é uma “capa” opcional. O protocolo pode rodar em texto claro (Porta 80) ou cifrado (Porta 443)
- **HTTP/3:** A segurança é **nativa**. O TLS 1.3 é parte do protocolo QUIC

Consequência Prática

Não existe HTTP/3 sem HTTPS. Todo tráfego HTTP/3 é, por definição, seguro e autenticado desde o primeiro byte enviado

Fluxo de Operação: HTTP/3 (QUIC)



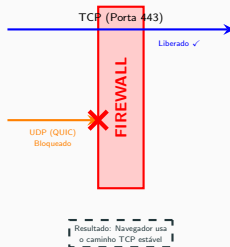
HTTP/3 vs. Firewalls: O Desafio do UDP

Muitas redes corporativas bloqueiam a porta **UDP 443** por segurança. O HTTP/3 resolve isso com um mecanismo de “tentativa e erro” transparente

Mecanismo de Descoberta (Alt-Svc)

Como o navegador sabe que pode usar HTTP/3?

1. **Primeiro Contato:** O cliente conecta via **TCP** (HTTP/1.1 ou 2)
2. **Anúncio:** O servidor responde com o cabeçalho: `Alt-Svc: h3=":443"; ma=86400`
3. **Upgrade:** Na próxima visita, o navegador tenta o **QUIC/UDP**



Resiliência (Fallback)

Se o firewall descartar os pacotes UDP, o navegador **volta para o HTTP/2 (TCP)** instantaneamente. O usuário não vê erro, apenas a conexão “padrão”

Anti-Ossificação

O QUIC criptografa seus próprios números de sequência, impedindo que firewalls “espertos” tentem modificar o tráfego, garantindo que o protocolo evolua sem interferência de dispositivos antigos

Resumo: Evolução e Comparativo HTTP

Característica	HTTP/1.0	HTTP/1.1	HTTP/2	HTTP/3
Lançamento	1996	1997	2015	2022
Transporte	TCP	TCP	TCP	UDP (QUIC)
Formato	Texto	Texto	Binário	Binário
Conexão TCP	1 por objeto	Persistente	Multiplexada	Nenhuma
Multiplexação	Não	Pipelining	Sim	Sim (Nativa)
HoL Blocking	Total	Aplicação	Transporte	Resolvido
Segurança	Opcional	Opcional	"Obrigatória"	TLS 1.3 Nativo
Compressão	Não	Não	HPACK	QPACK

Perspectiva Prática

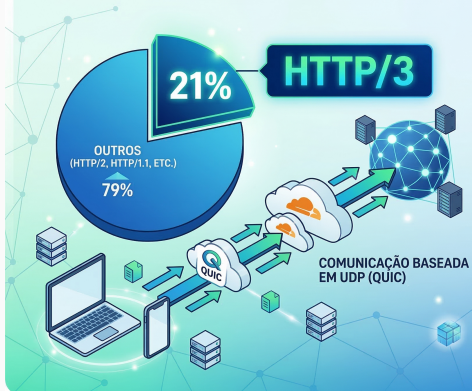
A migração para o HTTP/3 (QUIC) marca o fim da dependência do TCP para a Web. A complexidade de controle de fluxo saiu do *kernel* do SO e subiu para a aplicação (*Userspace*)

Ponto Chave

O HTTP/2 resolveu o bloqueio na camada de aplicação, mas o HTTP/3 foi necessário para resolver o bloqueio no transporte

Adoção do HTTP/3 na Internet

TRÁFEGO GLOBAL NA INTERNET (2025)



ADOÇÃO POR SITES DA WEB (MARÇO 2026)

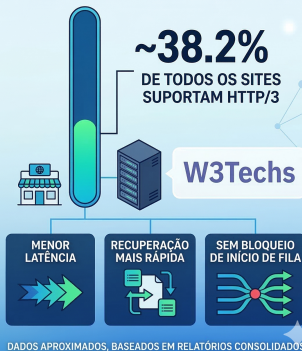


Imagem gerada por IA

Síntese: Revisão e Ponte para a Aula 3

Resumo Consolidado: Camada de Aplicação e HTTP

A comunicação entre processos em sistemas finais ocorre via troca de mensagens através de **Sockets** (Interface IP + Porta)

Arquiteturas e Conceitos

- **Paradigmas:** Cliente-Servidor (centralizado) vs. P2P (distribuído)
- **Endereçamento:** Identificado pelo par **IP + Porta** (Ex: HTTP: 80, HTTPS: 443)
- **Interface:** O *Socket* é a "porta" entre a aplicação e a rede

Protocolo HTTP

- **Modelo:** Requisição/Resposta
- **Estado:** Protocolo *stateless*. Uso de **Cookies** para gerenciar sessões
- **Desempenho:** **Web Caching** (Proxy) reduz latência e carga nos links

Evolução do Transporte Web

- v1.1:** Conexões persistentes e *Keep-Alive*. Sofre com HoL na aplicação
- v2:** **Binário e Multiplexado**. Uma única conexão TCP para múltiplos objetos
- v3:** **QUIC sobre UDP**. Elimina o HoL no transporte e integra TLS 1.3 nativo

Segurança

Originalmente implementada via **TLS/SSL** sobre o TCP. No HTTP/3, a criptografia é parte intrínseca do protocolo de transporte